SIMATS ENGINEERING

CO4 – ASSESSMENT TOOL 3: REVERSE ENGINEERING TASK

Cloud Computing and Big Data Analytics (CSA1558)

SET 1 – ALL 5 QUESTIONS ANSWERED IN DETAIL

CO4: Analyze big data characteristics and challenges across domains including security and application aspects.

Note: The question paper asks for reverse engineering of **probable** architectures. Therefore, the technologies mentioned below are reasoned architectural choices rather than claims about a specific real-world implementation.

1. Big data platform processing billions of website clicks, mobile interactions and customer transactions

Reverse-engineered architecture: The scale described—billions of events per day—strongly suggests a distributed, horizontally scalable architecture. A probable design is a data-ingestion layer feeding a distributed data lake/storage system, with separate batch and stream-processing paths and an analytics/BI layer.

1. Distributed storage: A distributed data lake based on HDFS or cloud object storage would be a likely foundation. Large volumes of raw clickstream, mobile and transaction data can be stored across many nodes/objects rather than on one server. Data can be partitioned by date, region, event type or customer-related dimensions to make queries efficient.

2. Ingestion framework: A distributed messaging system such as Apache Kafka would be a probable ingestion component. Website clicks, mobile events and transaction events can be published as separate topics. Producers send events continuously, while downstream consumers process them independently. This decouples event producers from analytics systems and allows the platform to absorb traffic spikes.

3. Structured and semi-structured data: Structured transaction data would likely be stored in relational/columnar analytical formats, while semi-structured click and mobile events could be stored as JSON/Avro-like records in the data lake. A metadata/catalog layer can describe schemas and partitions so analysts can discover datasets. Historical data can be converted into columnar formats such as Parquet for efficient analytical queries.

4. Batch processing: Large historical datasets would likely be processed using Hadoop MapReduce, Apache Spark, or another distributed batch-processing framework. Batch jobs can clean data, aggregate customer activity, generate reports and prepare datasets for machine-learning workflows.

5. Stream processing: For real-time analytics, probable engines include Apache Spark Structured Streaming, Apache Flink, or Kafka Streams. These systems can consume event streams, calculate rolling metrics, detect unusual activity and produce near-real-time dashboards.

6. Scalability and partitioning: Horizontal scaling is the key strategy. Instead of buying one extremely powerful machine, more workers are added as data volume grows. Storage and processing are partitioned by time, region, event type or another high-cardinality key. Good partitioning prevents a single node from becoming a bottleneck and allows parallel processing.

7. Analytics layer: A distributed query engine such as Hive, Spark SQL, Trino or Presto could provide SQL-style access over large datasets. Curated datasets can be exposed to analysts and machine-learning systems.

8. Dashboards and BI: Business-intelligence tools would connect to the analytical warehouse, query engine or prepared aggregate tables rather than directly processing raw events. Dashboards could display customer activity, transaction trends, conversion rates, regional performance and real-time KPIs.

Overall flow: Website/mobile/transaction sources → distributed event ingestion → raw data lake → batch + stream processing → curated analytical datasets → query/analytics layer → dashboards and BI systems.

Conclusion: The probable architecture uses distributed storage, parallel ingestion, partitioned datasets, separate real-time and batch processing paths, and an analytical serving layer. This design supports billions of events while allowing both historical analysis and real-time business reporting.

2. E-commerce platform tracking carts, abandoned checkouts, product views and payment behavior

Reverse-engineered architecture: The platform requires real-time personalization while combining high-volume browsing events with reliable transactional records. A probable architecture therefore separates event streaming, transactional storage, customer profiles, analytics and recommendation services while continuously synchronizing them.

1. Event streaming: Apache Kafka or a similar distributed messaging platform would likely capture product views, cart additions, checkout events and payment-related events. Separate topics can represent different event categories. Partitioning allows multiple consumers to process events in parallel.

2. Customer profiling database: A NoSQL database such as Cassandra, HBase, DynamoDB or MongoDB could maintain rapidly changing customer-profile information. A relational database would remain appropriate for strongly structured order/payment information.

3. Session and clickstream synchronization: User sessions can be represented using a session ID or customer ID. Events from web/mobile applications are published to the event stream. Stream processors join or aggregate recent events to update customer profiles and personalization features. Transaction records can be linked using customer/order identifiers.

4. Real-time personalization: A stream-processing engine can calculate features such as recently viewed products, cart contents, browsing frequency and abandoned checkout status. The recommendation service can use these features to select products or offers dynamically.

5. Caching: A low-latency cache such as Redis could store frequently accessed customer profiles, recommendation results, session information and product metadata. Caching reduces repeated database queries and improves response time for high-traffic pages.

6. Distributed query and analytics: A distributed SQL/query engine such as Trino, Presto, Spark SQL or Hive could query historical browsing and transaction data. This supports customer segmentation, sales analysis and recommendation-model preparation.

7. Machine-learning workflow: Historical clickstream, transaction and customer data can be cleaned and transformed into training features. Models can be trained for product recommendation, customer segmentation, churn/abandonment prediction or next-best-action prediction. Trained models can then be deployed to score new events.

8. Merging structured and semi-structured data: Structured order/payment data can be joined with semi-structured browsing events using stable identifiers such as customer ID, session ID, product ID and timestamps. A curated analytics layer can store a unified customer-event view.

9. Scalability and fault tolerance: Kafka partitions allow parallel event consumption. Replication protects event data from broker failures. Distributed databases replicate customer data across nodes, while stateless recommendation services can scale horizontally. Processing checkpoints and replayable event streams help recover from failures.

10. KPI dashboards: Aggregated datasets can feed BI dashboards showing conversion rate, cart abandonment, revenue, product performance, customer activity and recommendation performance. Real-time metrics can be served from streaming aggregates, while historical KPIs can come from the analytical warehouse/data lake.

Overall flow: Web/mobile events + transaction systems → Kafka/event streaming → stream processing → session/profile updates + cache → recommendation service; simultaneously → data lake/warehouse → distributed queries + ML → BI dashboards.

Conclusion: The likely design separates fast transactional operations from large-scale analytics while connecting them through event streaming and shared identifiers. This provides low-latency personalization without forcing the transactional database to perform all analytical work.

3. Big data healthcare system processing patient histories, diagnostic images, wearable streams and prescriptions

Reverse-engineered architecture: Healthcare data is highly heterogeneous. Patient histories and prescriptions are structured, diagnostic images are large unstructured objects, and wearable devices generate continuous time-series streams. A hybrid architecture is therefore likely.

1. Hybrid storage: A relational database would store structured information such as patient demographics, appointments, prescriptions and laboratory records. A distributed object store/HDFS-based data lake would store diagnostic images, documents, raw sensor streams and other large datasets. A NoSQL/time-series store could support rapidly arriving wearable measurements.

2. Data integration: Hospitals, laboratories and insurance systems can exchange information through healthcare interoperability interfaces and controlled ETL/data-integration pipelines. A central integration layer validates formats, maps fields and associates records using appropriate patient identifiers.

3. Streaming: Wearable sensor data can be sent through a messaging system such as Kafka. A stream processor can calculate real-time metrics, detect abnormal measurements and trigger monitoring workflows.

4. Machine learning: Historical patient records, diagnostic information and sensor data can be transformed into training datasets. ML models could support disease-risk prediction, anomaly detection, patient deterioration alerts or population-level risk analysis. Model outputs should be treated as decision support rather than automatically replacing clinical judgment.

5. Privacy and encryption: Healthcare information requires strong access controls. Data should be encrypted during transmission and at rest. Role-based access can ensure that clinicians, analysts, administrators and external users receive only the information required for their roles.

6. Compliance and auditing: The platform would need appropriate healthcare privacy and compliance controls based on the applicable jurisdiction and organization. Access logs, audit trails, data-retention policies and controlled sharing help demonstrate who accessed sensitive information and when.

7. Analytics pipeline: Source systems → secure integration/ETL → raw healthcare data lake → cleansing and standardization → analytical datasets → distributed processing/ML → clinical dashboards and population-health analytics.

8. Clinical decision support: Curated patient and population datasets can be queried to identify trends, risk factors and patterns. Real-time streams can support monitoring, while historical datasets support research and population-health studies.

Conclusion: The probable architecture separates storage according to data characteristics while connecting all sources through controlled integration. Encryption, access control and auditing protect sensitive data, while distributed processing enables large-scale clinical and population analytics.

4. Smart city platform gathering traffic sensors, CCTV metadata, weather and public transport activity

Reverse-engineered architecture: A smart-city system receives continuous data from geographically distributed sources. A likely design therefore combines edge computing, distributed messaging, time-series/geospatial storage, stream processing and a centralized analytics platform.

1. Edge computing: Traffic sensors, cameras and transport devices can perform initial filtering or aggregation near the source. For example, CCTV systems may send metadata such as vehicle counts or detected events instead of continuously transferring every raw video frame to the central platform. This reduces bandwidth usage.

2. Distributed messaging: A platform such as Kafka or another event-bus system can collect traffic, weather and transport events. Topics can separate event categories, while partitions distribute processing across multiple consumers.

3. Centralized storage: A data lake can retain historical sensor data and large files. Time-series databases are suitable for measurements indexed by time, while geospatial databases support location-based queries.

4. Geospatial analytics: Traffic events can be associated with road segments, coordinates or geographic zones. Stream-processing jobs can calculate congestion levels, vehicle counts and travel-time estimates for each area.

5. Congestion prediction: Historical traffic patterns can be combined with current sensor data and weather information. Stream processing provides current conditions, while ML or batch analytics can identify patterns and predict future congestion.

6. Time-series and location-aware databases: A time-series database can efficiently query sensor measurements over time. A geospatially capable relational/NoSQL system can support queries such as nearby incidents, affected road segments or vehicle activity within a geographic region.

7. Batch + real-time analytics: Real-time processing supports operational decisions such as congestion alerts and transport monitoring. Batch processing analyzes historical data for road planning, infrastructure investment, traffic-pattern studies and long-term urban planning.

8. Security and access control: Citizen and infrastructure data should be protected using authentication, authorization, encryption, network segmentation and audit logging. Access should follow least-privilege principles, especially for sensitive camera metadata and critical infrastructure information.

9. Visualization: Operational dashboards can show live maps, congestion levels, incidents, public-transport status and weather conditions. Historical dashboards can compare traffic patterns by day, location and time period.

Overall flow: Sensors/CCTV/transport/weather → edge filtering → event messaging → stream processing → time-series/geospatial analytics; simultaneously → data lake → batch analytics → planning insights → operational and management dashboards.

Conclusion: Combining edge processing with distributed messaging reduces latency and bandwidth pressure, while centralized storage and batch processing provide the historical foundation required for city planning.

5. Healthcare analytics system integrating patient records, wearable readings, lab reports and appointment histories

Reverse-engineered architecture: This scenario requires a hybrid healthcare data platform because patient records and appointments are structured, laboratory reports may contain structured and document-like content, and wearable readings are continuous time-series data.

1. Hybrid storage: Use a relational database for structured patient, appointment and transactional information. Store large documents and raw datasets in a distributed data lake/object storage. Wearable measurements can be stored in a time-series or scalable NoSQL system for efficient time-based access.

2. ETL and interoperability: ETL pipelines can extract data from hospitals and clinics, transform inconsistent fields into common representations, validate records and load them into the analytics platform. Healthcare interoperability standards can be used where applicable so different hospital/clinic systems can exchange information consistently.

3. Real-time monitoring: Wearable devices can send events through a streaming layer such as Kafka. Stream processors can calculate recent heart rate, activity or other available metrics and detect patterns that require attention.

4. Predictive analytics: Historical patient records, laboratory information and wearable measurements can be transformed into features for machine-learning models. Possible applications include risk scoring, deterioration prediction and treatment-support analytics. The models should be validated and governed appropriately before being used in clinical workflows.

5. Distributed processing: Population-level studies may involve millions of records. Distributed processing frameworks such as Spark or Hadoop MapReduce can divide the workload across multiple nodes, allowing large datasets to be processed in parallel.

6. Encryption and compliance: Sensitive medical information should be encrypted during transmission and storage. Role-based access controls should restrict access according to job responsibilities. Audit logging should record access and important data operations.

7. Data governance: Data should have defined ownership, retention rules, access policies and lineage. Patient identifiers should be handled carefully, and analytical datasets can use controlled de-identification or pseudonymization when appropriate for research.

8. Machine-learning support: Models can combine multiple data sources to generate risk scores or other decision-support outputs. A robust pipeline should include data quality checks, feature preparation, model validation, monitoring and controlled deployment.

9. Analytics and visualization: A curated analytical layer can provide dashboards for population trends, appointment patterns, lab trends and monitoring indicators. Real-time dashboards can consume streaming aggregates, while historical reports can use the analytical warehouse/data lake.

Overall flow: Patient/clinic systems + wearables → secure ETL/interoperability + event streaming → hybrid storage/data lake → distributed processing + ML → governed analytics layer → clinical/population dashboards.

Conclusion: The likely solution combines relational, distributed and time-series storage with secure integration and distributed processing. This allows the platform to handle heterogeneous medical data while supporting both real-time monitoring and large-scale population analytics.

Final note: These are reverse-engineered probable architectures based on the scenarios in SET 1. They should be presented as reasoned inferences rather than as confirmed implementations of a named organization.